

Enabling Window-Based Analytics for Property Graphs

Domagoj Vrnoc

Riccardo Tommas

May 2026

Abstract

This note organizes several possible meanings of window functions in property graph query languages. The central distinction is the domain over which the computation iterates. A SQL-style window iterates over rows of a binding table. A path window iterates over the ordered elements of an already bound path. A graph window evaluates a topological neighborhood around a focus binding. A walk window constructs a path while computing, using a transition pattern. A path-scoped pattern window matches a local subgraph pattern at each position of an already bound path. These constructs have different semantics, cardinality behavior, optimization challenges, and use cases.

Contents

1	Preliminaries	2
1.1	Window-Function in SQL	2
1.2	Examples of SQL window functions	2
1.3	Filtering window results with <code>QUALIFY</code>	5
1.4	Window-function invariants.	6
1.5	Common Table Expressions	7
1.6	Graphs	8
1.6.1	Data	8
1.6.2	Querying	9
2	SQL-LIKE Window Functions for Property Graph Queries	11
2.1	Semantics	11
2.2	Partitioning	12
2.3	Ranking	14
2.4	Framing	14
2.5	Path-element framing	16
2.6	CTEs and graph bindings windows.	21
2.7	Strengths	22
2.8	Invariants	22
2.9	Limitations	22

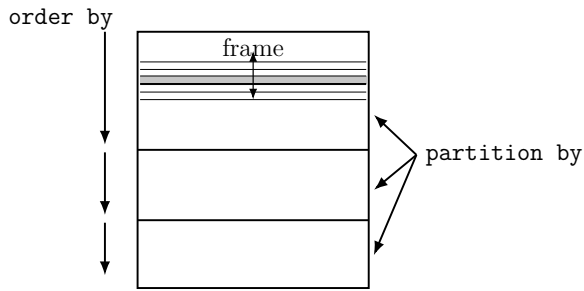


Figure 1: Window function concepts: partitioning, ordering, framing. The current (gray) row can access rows in its frame. The frame of a tuple can only encompass tuples from that partition.

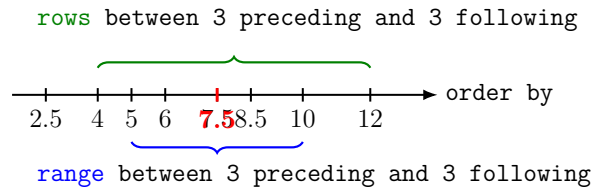


Figure 2: Illustration of the **range** and **rows** modes for framing. Each tick represents the value of a tuple's **order by** expression.

Figure 1: Leis et al.

1 Preliminaries

1.1 Window-Function in SQL

Window functions evaluate expressions over a relation without collapsing that relation. Unlike ordinary aggregation, a window operator preserves the input tuples and appends new attributes computed from related tuples. Its semantics is usually decomposed into three orthogonal steps: partitioning, ordering, and framing.

Following Figure 1 Partitioning divides the input relation into independent groups; ordering defines a logical order inside each partition; framing selects, for each current tuple, the subset of nearby tuples that is visible to the window function.

The partitioning step is not a **GROUP BY**. It does not reduce each group to one tuple. Instead, it creates logical partitions while preserving the individual tuples. If no partitioning clause is specified, the whole input relation forms a single partition. The ordering step is also local to the window operator: it determines the order used for ranking, navigation, and frame construction, but it does not necessarily determine the final output order.

The frame is defined relative to the current tuple. In **ROWS** mode, the frame is specified by tuple offsets, such as three preceding rows and the current row. In **RANGE** mode, the frame is specified by offsets over the **ORDER BY** value. Thus, framing restricts the tuples visible to frame-sensitive functions such as **sum**, **avg**, **min**, **max**, **first_value**, and **last_value**. Other functions, such as **rank**, **row_number**, **lead**, and **lag**, are evaluated over the partition and ignore the frame.

1.2 Examples of SQL window functions

The examples below assume a relation **Transfer** with schema:

Listing 1: Example relational input used for SQL window-function examples.

```
Transfer(
  source    VARCHAR,
  target    VARCHAR,
  department VARCHAR,
  time      TIMESTAMP,
  amount    NUMERIC,
  currency  VARCHAR
)
```

Partitioning. The following query ranks transfers independently inside each currency partition.

Listing 2: Ranking rows inside currency partitions.

```
SELECT
  source,
  target,
  currency,
  amount,
  RANK() OVER (
    PARTITION BY currency
    ORDER BY amount DESC
  ) AS amount_rank
FROM Transfer;
```

Here, `PARTITION BY currency` splits the input relation into one partition per currency. The ranking is computed independently inside each partition.

Window aggregation over a partition. A window aggregate preserves the input rows, unlike `GROUP BY`. The following query computes the total transferred amount per currency while keeping one output row per transfer.

Listing 3: Window aggregation over each currency partition.

```
SELECT
  source,
  target,
  currency,
  amount,
  SUM(amount) OVER (
    PARTITION BY currency
  ) AS total_amount_per_currency
FROM Transfer;
```

This differs from ordinary aggregation:

Listing 4: Ordinary aggregation collapses rows.

```
SELECT
  currency,
  SUM(amount) AS total_amount_per_currency
FROM Transfer
GROUP BY currency;
```

The query in Listing 3 preserves all transfer rows. The query in Listing 4 returns one row per currency.

Cumulative aggregation. When an `ORDER BY` clause is present, a window aggregate can compute a running value.

Listing 5: Cumulative transferred amount per currency.

```
SELECT
  source,
  target,
  currency,
  time,
  amount,
  SUM(amount) OVER (
    PARTITION BY currency
```

```

ORDER BY time
ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
) AS cumulative_amount
FROM Transfer;

```

For each row, the frame contains all previous rows in the same currency partition, including the current row.

ROWS framing. A ROWS frame is defined by physical tuple offsets in the ordered partition.

Listing 6: Moving average over the current row and two previous rows.

```

SELECT
  source,
  target,
  currency,
  time,
  amount,
  AVG(amount) OVER (
    PARTITION BY currency
    ORDER BY time
    ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
  ) AS moving_avg_3_rows
FROM Transfer;

```

For each current row, the frame contains at most three tuples: the current row and the two preceding rows in the same partition.

RANGE framing. A RANGE frame is defined by values of the ORDER BY expression, not by tuple offsets.

Listing 7: Thirty-day rolling sum using a RANGE frame.

```

SELECT
  source,
  target,
  currency,
  time,
  amount,
  SUM(amount) OVER (
    PARTITION BY currency
    ORDER BY time
    RANGE BETWEEN INTERVAL '30' DAY PRECEDING
    AND CURRENT ROW
  ) AS rolling_30d_amount
FROM Transfer;

```

For each transfer, the frame contains transfers in the same currency partition whose timestamps fall within the previous thirty days.

GROUPS framing. A GROUPS frame is defined over peer groups. Rows are peers when they have the same ORDER BY value.

Listing 8: Frame over peer groups with equal ordered values.

```

SELECT
  source,
  target,
  currency,

```

```

amount,
COUNT(*) OVER (
  PARTITION BY currency
  ORDER BY amount
  GROUPS BETWEEN 1 PRECEDING AND CURRENT ROW
) AS count_current_and_previous_amount_group
FROM Transfer;

```

This frame contains the current peer group and the immediately preceding peer group. If several transfers have the same `amount`, they belong to the same peer group.

Navigation functions. Window functions can also access neighboring tuples.

Listing 9: Accessing the previous transfer in each currency partition.

```

SELECT
  source,
  target,
  currency,
  time,
  amount,
  amount - LAG(amount) OVER (
    PARTITION BY currency
    ORDER BY time
  ) AS difference_from_previous
FROM Transfer;

```

The function `LAG` reads a value from a previous tuple in the same partition.

1.3 Filtering window results with `QUALIFY`

Window functions are evaluated after ordinary row filtering. Therefore, a window expression cannot generally be used directly in a `WHERE` clause, because `WHERE` is evaluated before the window attribute exists. The `QUALIFY` clause addresses this problem by providing a filtering phase after window-function evaluation.

The typical structure is:

Listing 10: Filtering rows after window evaluation using `QUALIFY`.

```

SELECT
  source,
  target,
  currency,
  amount,
  RANK() OVER (
    PARTITION BY currency
    ORDER BY amount DESC
  ) AS amount_rank
FROM Transfer
QUALIFY amount_rank <= 2;

```

The query in Listing 10 first computes the rank of each transfer inside its currency partition. The `QUALIFY` clause then keeps only the two highest-ranked transfers per currency. This is different from `WHERE`, which filters rows before ranking.

Without `QUALIFY`, the same query requires an additional query block:

Listing 11: Equivalent formulation using a derived table instead of `QUALIFY`.

```

SELECT *

```

Clause	Phase	Can refer to window output?
WHERE	before window evaluation	no
HAVING	after grouping	no, unless nested
QUALIFY	after window evaluation	yes
ORDER BY	final output ordering	yes

Table 1: table

Logical position of **QUALIFY** with respect to window-function evaluation.

```
FROM (
  SELECT
    source,
    target,
    currency,
    amount,
    RANK() OVER (
      PARTITION BY currency
      ORDER BY amount DESC
    ) AS amount_rank
  FROM Transfer
) AS RankedTransfers
WHERE amount_rank <= 2;
```

Thus, **QUALIFY** is syntactic support for predicates over windowed attributes. It does not change the semantics of window functions. It only makes explicit that the predicate is evaluated after the window operator.

Logical role. The logical distinction is:

For graph-window semantics, **QUALIFY** is useful whenever a query filters on a value computed by a graph window. For example, if a path ranking is computed over a relation of path bindings, the filter on the rank belongs after the window phase:

```
MATCH p = (a:Account)-[:TRANSFER*]->(b:Account)
RETURN
  a.name AS source,
  b.name AS target,
  p,
  RANK() OVER (
    PARTITION BY a.name, b.name
    ORDER BY length(p)
  ) AS path_rank
QUALIFY path_rank = 1
```

Listing 12: Graph-style use of **QUALIFY** over a path ranking.

Listing 12 keeps only the shortest path or paths per source-target pair, depending on how ties are handled by **RANK**. The important point is that **path_rank** is not available during pattern matching. It becomes available only after the window operator has decorated the binding tuples.

1.4 Window-function invariants.

A row-based window operator satisfies the following invariants.

- I1 **Tuple preservation.** The window operator does not remove, duplicate, or collapse input tuples. It appends computed attributes to each input tuple.
- I2 **Partition coverage.** Every input tuple belongs to exactly one partition. If no **PARTITION BY** clause is given, all tuples belong to one global partition.

Feature	Example	Meaning
Partitioning	Listing 2	split rows into independent groups
Aggregation	Listing 3	compute aggregate without collapsing rows
Cumulative frame	Listing 5	aggregate from partition start to current row
ROWS frame	Listing 6	frame by tuple offsets
RANGE frame	Listing 7	frame by ordered-value intervals
GROUPS frame	Listing 8	frame by peer groups
Navigation	Listing 9	access neighboring tuples

Table 2: Summary of the SQL window-function examples and their corresponding listings.

- I3 **Partition disjointness.** Partitions do not overlap. A tuple cannot belong to two different partitions for the same window specification.
- I4 **Partition-local access.** A tuple can access only tuples in its own partition. Its frame is always a subset of its partition.
- I5 **Ordering locality.** `ORDER BY` defines a logical order only inside each partition. It does not by itself prescribe the final output order of the query.
- I6 **Frame relativization.** The frame is computed relative to the current tuple and the partition-local order. Each tuple may therefore have a different frame.
- I7 **Frame containment.** For every tuple t , $\text{frame}(t) \subseteq \text{partition}(t)$. (This is the same as I4)
- I8 **Peer consistency.** Tuples with the same `ORDER BY` value inside a partition are peers. Peer-sensitive functions such as `rank` assign the same rank to peers.

1.5 Common Table Expressions

A Common Table Expression (CTE) names an intermediate relation inside a query. It is introduced by a `WITH` clause and can be referenced by subsequent query blocks. CTEs are useful for readability, modularity, and for making logical evaluation phases explicit.

A non-recursive CTE has the following shape:

Listing 13: Basic Common Table Expression syntax.

```
WITH NamedRelation AS (
  SELECT ...
  FROM ...
  WHERE ...
)
SELECT ...
FROM NamedRelation;
```

CTEs are particularly useful with window functions because predicates over window results require a later query phase. For example:

Listing 14: Using a CTE to filter rows after computing a window function.

```
WITH RankedTransfers AS (
  SELECT
    source,
    target,
    currency,
    amount,
    RANK() OVER (
      PARTITION BY currency
      ORDER BY amount DESC
    ) AS amount_rank
```

```

    FROM Transfer
)
SELECT
    source,
    target,
    currency,
    amount,
    amount_rank
FROM RankedTransfers
WHERE amount_rank <= 2;

```

The CTE in Listing 14 materializes the logical result of the window phase: each transfer row has been extended with `amount_rank`. The outer query can then filter on that attribute.

Recursive CTEs. A recursive CTE defines a relation by repeatedly applying a recursive rule until a fixpoint or a stopping condition is reached. Its structure contains a base case and a recursive step:

Listing 15: Schematic recursive CTE structure.

```

WITH RECURSIVE Reachable(src, dst, depth) AS (
    -- base case
    SELECT src, dst, 1
    FROM Edge

    UNION ALL

    -- recursive step
    SELECT r.src, e.dst, r.depth + 1
    FROM Reachable r
    JOIN Edge e ON r.dst = e.src
    WHERE r.depth < 5
)
SELECT *
FROM Reachable;

```

Recursive CTEs are relevant to graph queries because reachability and bounded path expansion can be expressed recursively. However, they still produce relations. They do not, by themselves, define topological frames. For topological windows, the analogous construct would be graph-valued rather than table-valued.

1.6 Graphs

1.6.1 Data

Assume the graph contains the following Nodes:

add currency does

```

(:Account {name:'Alice', domain:'Research', salary:120000})
(:Account {name:'Bob', domain:'Research', salary:95000})
(:Account {name:'Carol', domain:'Engineering', salary:105000})
(:Account {name:'Dana', domain:'Engineering', salary:115000})
(:Account {name:'Eve', domain:'Sales', salary:90000})

```

Assume the transfer graph is the one used in the previous examples, and assume the following currency nodes and relationships:

```
(:Currency {code:'EUR', risk:1}) -[:ISSUED_FOR]-> (:Account {name:'Bob'})
(:Currency {code:'EUR', risk:1}) -[:ISSUED_FOR]-> (:Account {name:'Carol'})
(:Currency {code:'USD', risk:2}) -[:ISSUED_FOR]-> (:Account {name:'Carol'})
(:Currency {code:'USD', risk:2}) -[:ISSUED_FOR]-> (:Account {name:'Eve'})
(:Currency {code:'EUR', risk:1}) -[:ISSUED_FOR]-> (:Account {name:'Dana'})
(:Currency {code:'GBP', risk:3}) -[:ISSUED_FOR]-> (:Account {name:'Dana'})
(:Currency {code:'USD', risk:2}) -[:ISSUED_FOR]-> (:Account {name:'Dana'})
```

The graph contains the following TRANSFER edges (omitting some node attributes):

```
(:Account {name:'Alice'}) -[:TRANSFER {transferNo: 1, time: datetime('2026-01-05T09
:15:00Z'), amount: 2500, currency: 'EUR'}]-> (:Account {name:'Bob'})
(:Account {name:'Alice'}) -[:TRANSFER {transferNo: 2, time: datetime('2026-01-06T16
:20:00Z'), amount: 3000, currency: 'USD'}]-> (:Account {name:'Eve'})
(:Account {name:'Bob'}) -[:TRANSFER {transferNo: 3, time: datetime('2026-01-07T14
:30:00Z'), amount: 1200, currency: 'EUR'}]-> (:Account {name:'Carol'})
(:Account {name:'Carol'}) -[:TRANSFER {transferNo: 4, time: datetime('2026-01-09T11
:45:00Z'), amount: 800, currency: 'EUR'}]-> (:Account {name:'Dana'})
(:Account {name:'Eve'}) -[:TRANSFER {transferNo: 5, time: datetime('2026-01-10T10
:05:00Z'), amount: 1500, currency: 'USD'}]-> (:Account {name:'Dana'})
```

1.6.2 Querying

Pattern matching A property graph query typically starts by matching graph patterns. The result of a match is not merely a graph object. It is a stream or table of bindings.

```
MATCH (u:Account)-[r:TRANSFER]->(v:Account)
RETURN u.name, u.domain, type(r), v.name, v.domain
```

A possible binding table is:

<i>u.name</i>	<i>u.domain</i>	<i>r</i>	<i>v.name</i>	<i>v.domain</i>
Alice	Research	TRANSFER	Bob	Research
Bob	Research	TRANSFER	Carol	Engineering
Carol	Engineering	TRANSFER	Dana	Engineering
Alice	Research	TRANSFER	Eve	Sales
Eve	Sales	TRANSFER	Dana	Engineering

```
MATCH p=(u:Account)-[:TRANSFER*]->(v:Account)
RETURN u.name, u.domain, v.name, v.domain, p
```

Paths A property graph query can take the form of a path query. The result is still a binding table, but one of the bindings is a path value. A path binding is richer than a node or edge binding. If $p = v_0 \xrightarrow{e_1} v_1 \xrightarrow{e_2} v_2 \cdots \xrightarrow{e_k} v_k$, then p has an internal sequence of nodes, edges, and positions.

<i>u.name</i>	<i>u.domain</i>	<i>v.name</i>	<i>v.domain</i>	<i>p</i>
Alice	Research	Bob	Research	Alice $\rightarrow$ Bob
Bob	Research	Carol	Engineering	Bob $\rightarrow$ Carol
Carol	Engineering	Dana	Engineering	Carol $\rightarrow$ Dana
Alice	Research	Eve	Sales	Alice $\rightarrow$ Eve
Eve	Sales	Dana	Engineering	Eve $\rightarrow$ Dana
Alice	Research	Carol	Engineering	Alice $\rightarrow$ Bob $\rightarrow$ Carol
Bob	Research	Dana	Engineering	Bob $\rightarrow$ Carol $\rightarrow$ Dana
Alice	Research	Dana	Engineering	Alice $\rightarrow$ Eve $\rightarrow$ Dana
Alice	Research	Dana	Engineering	Alice $\rightarrow$ Bob $\rightarrow$ Carol $\rightarrow$ Dana

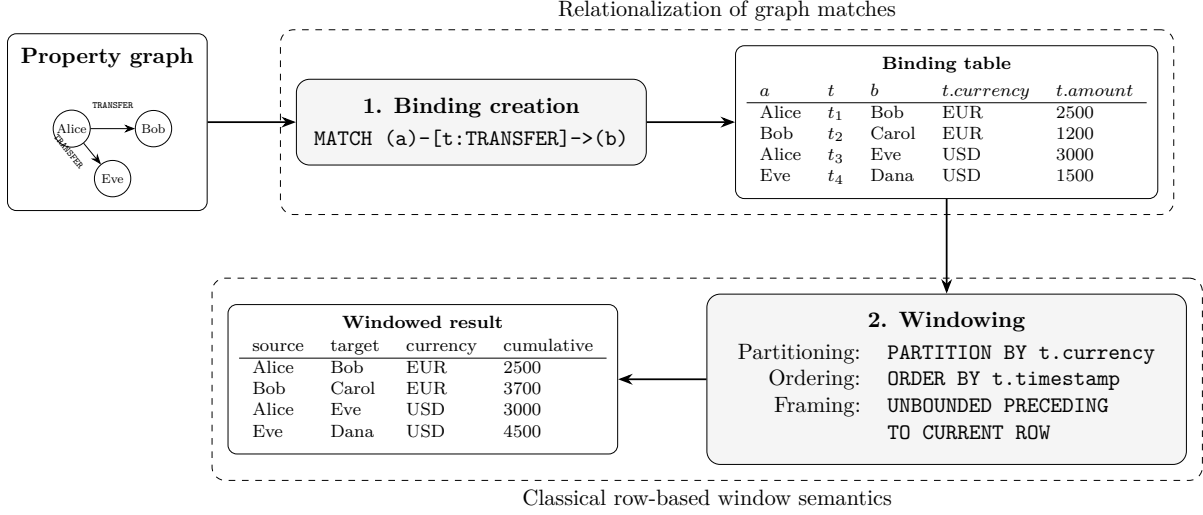


Figure 2: Logical execution of Alternative 1. Pattern matching first creates a binding relation; windowing is then evaluated over that relation.

2 SQL-LIKE Window Functions for Property Graph Queries

This is the closest analogue to SQL window functions. A graph pattern is evaluated first and produces a relation of bindings. Each binding may contain scalar values, node objects, edge objects, or path objects. The window operator is then applied to this relation using ordinary row-based semantics: partitioning, ordering, and framing operate over bindings, not directly over the graph. If a binding contains a path value p , the path is treated as an atomic value inside the binding. The window may use path-global expressions such as `length(p)`, `startNode(p)`, `endNode(p)`, `totalAmount(p)`, or `endTime(p)`, but it does not automatically iterate over `nodes(p)` or `relationships(p)`.

2.1 Semantics

Let R be the binding table produced by the preceding graph query. A row window computes a value for each binding $r \in R$ using a frame selected from the same relation R .

$$\text{win}(r) = F(\text{frame}(r, R)).$$

The frame is always a set of bindings. If a binding contains a path object, then the binding is still the unit of windowing. The path object is a value carried by the binding, not the iteration domain of the frame.

The number of output bindings is normally the same as the number of input bindings, unless a subsequent **WHERE**, **WITH**, **QUALIFY**-style clause, or projection changes it.

TODO: Give a formal definition of syntax/semantics with all the considered elements.

2.2 Partitioning

The partition key can be a node object, an edge object, a path object, a label, a property, or any scalar expression.

Partitioning by a node object. The first example partitions by the source account node **a**. All outgoing transfer rows from the same account belong to the same partition.

```
MATCH (a:Account)-[t:TRANSFER]->(b:Account)
RETURN a.name AS source, b.name AS target,
       t.amount AS amount, t.currency AS currency,
       rank() OVER (PARTITION BY a ORDER BY t.amount DESC)
       AS rank_per_source_account
```

Listing 16: Partitioning transfer rows by the source account node.

Table 3 shows a possible result.

a	b	amount	currency	rank_per_source_account
Alice	Eve	3000	USD	1
Alice	Bob	2500	EUR	2
Bob	Carol	1200	EUR	1
Carol	Dana	800	EUR	1
Eve	Dana	1500	USD	1

Table 3: Result of the node-object partition query in Listing 16.

The partition key is the account node object **a**. Therefore, Alice’s two outgoing transfers are ranked together, while Bob, Carol, and Eve each form separate one-row partitions.

Partitioning by an edge object. The second example partitions by the transfer edge object **t**. The query joins each transfer with the currencies issued for its target account **b**. If the target account has several issued currencies, the same transfer edge produces several rows.

```
MATCH (a:Account)-[t:TRANSFER]->(b:Account),
      (b)-[:ISSUED_FOR]-(c:Currency)
RETURN t, a.name AS src, b.name AS trgt,
       c.code AS currency, c.risk AS risk,
       sum(c.risk) OVER (PARTITION BY t ORDER BY c.code)
       AS cumul_risk
```

Listing 17: Partitioning joined currency rows by the transfer edge.

transfer	src	trgt	currency	risk	cumul_risk
<i>t</i> ₁ :TRANSFER	Alice	Bob	EUR	1	1
<i>t</i> ₂ :TRANSFER	Bob	Carol	EUR	1	1
<i>t</i> ₂ :TRANSFER	Bob	Carol	USD	2	3
<i>t</i> ₃ :TRANSFER	Carol	Dana	EUR	1	1
<i>t</i> ₃ :TRANSFER	Carol	Dana	GBP	3	4
<i>t</i> ₃ :TRANSFER	Carol	Dana	USD	2	6
<i>t</i> ₄ :TRANSFER	Alice	Eve	USD	2	2
<i>t</i> ₅ :TRANSFER	Eve	Dana	EUR	1	1
<i>t</i> ₅ :TRANSFER	Eve	Dana	GBP	3	4
<i>t</i> ₅ :TRANSFER	Eve	Dana	USD	2	6

Table 4: Result of the edge-object partition query in Listing 17.

The edge object t does not create multiple rows by itself. The multiplicity is introduced by the join with **Currency**. If the target account b is associated with several currencies, the same transfer edge t appears in several binding rows. Partitioning by t then groups those rows into one edge-object partition.

For example, the edge **Carol** $\rightarrow$ **Dana** forms one partition. Since Dana has three issued currencies, that partition contains three rows ordered by $c.code$: EUR, GBP, and USD. The cumulative risk values are therefore 1, $1 + 3 = 4$, and $1 + 3 + 2 = 6$.

Partitioning by a property or a scalar. This is straightforward because the partition key behaves as a column in a relation.

```
MATCH (u:Account)-[f:TRANSFER]->(v:Account)
RETURN u.name, u.domain, v.name, v.domain, v.score,
       rank() OVER (PARTITION BY u.domain ORDER BY v.score DESC)
       AS neighbor_rank
```

Listing 18: Partitioning transfer rows by a scalar property.

<i>u.name</i>	<i>u.domain</i>	<i>v.name</i>	<i>v.domain</i>	<i>v.score</i>	<i>neighbor_rank</i>
Alice	Research	Carol	Engineering	91	1
Bob	Research	Dana	Engineering	84	2
Alice	Research	Bob	Research	72	3
Dana	Engineering	Frank	Sales	95	1
Carol	Engineering	Alice	Research	88	2
Eve	Sales	Carol	Engineering	91	1

Table 5: Result of the scalar-property partition query in Listing 18.

Using a path object as a partition key. This example uses the bound path object p itself as the partition key. This is a special case of row-based windowing: rows are grouped by identical path identity. The window does not iterate over the internal elements of p . Instead, p is an atomic binding value, and all rows that share the same path binding belong to the same partition.

```
MATCH p = (a:Account)-[:TRANSFER*]->(b:Account),
       (b)-[:ISSUED_FOR]-(c:Currency)
RETURN p, a.name AS source, b.name AS target, c.code AS currency,
       sum(c.risk) OVER (PARTITION BY p ORDER BY c.code)
       AS cumulative_currency_risk
```

Listing 19: Using a path object as the partition key.

The query first matches transfer paths from an account a to an account b . It then joins each target account b with the currencies issued for that account. The window partition key is the path p . Therefore, if the same path is associated with several currencies, the window function accumulates currency risk values within that path-identity partition.

For a target account with multiple currencies, the rows are ordered by $c.code$. For example, for **Dana**, the order is EUR, GBP, USD; the corresponding cumulative risk values are 1, $1 + 3 = 4$, and $1 + 3 + 2 = 6$.

The important point is that this query partitions by path identity, not by the endpoint pair ($source, target$). Thus, the two paths from Alice to Dana, **Alice** $\rightarrow$ **Eve** $\rightarrow$ **Dana** and **Alice** $\rightarrow$ **Bob** $\rightarrow$ **Carol** $\rightarrow$ **Dana**, are placed in two different partitions. Even though both paths have the same source and target, they are distinct path objects, so the cumulative currency risk is computed independently for each path. This case should be distinguished from later examples where the input relation consists of path bindings but the partition key is an endpoint pair such as (a, b) .

p	source	target	currency	cumulative_currency_risk
Alice → Bob	Alice	Bob	EUR	1
Bob → Carol	Bob	Carol	EUR	1
Bob → Carol	Bob	Carol	USD	3
Carol → Dana	Carol	Dana	EUR	1
Carol → Dana	Carol	Dana	GBP	4
Carol → Dana	Carol	Dana	USD	6
Alice → Eve	Alice	Eve	USD	2
Eve → Dana	Eve	Dana	EUR	1
Eve → Dana	Eve	Dana	GBP	4
Eve → Dana	Eve	Dana	USD	6
Alice → Bob → Carol	Alice	Carol	EUR	1
Alice → Bob → Carol	Alice	Carol	USD	3
Bob → Carol → Dana	Bob	Dana	EUR	1
Bob → Carol → Dana	Bob	Dana	GBP	4
Bob → Carol → Dana	Bob	Dana	USD	6
Alice → Eve → Dana	Alice	Dana	EUR	1
Alice → Eve → Dana	Alice	Dana	GBP	4
Alice → Eve → Dana	Alice	Dana	USD	6
Alice → Bob → Carol → Dana	Alice	Dana	EUR	1
Alice → Bob → Carol → Dana	Alice	Dana	GBP	4
Alice → Bob → Carol → Dana	Alice	Dana	USD	6

Table 6: Result of the path-object partition query in Listing 19.

2.3 Ranking

Since Alternative 1 applies windowing to a binding relation, ranking is straightforward when the binding attributes are scalar values, node objects, or edge objects. The distinctive case is ranking path bindings. Here the path is an atomic binding value, so the ranking expression must be path-global.

Examples of admissible ordering expressions are:

- ORDER BY length(p)
- ORDER BY startNode(p) or ORDER BY endNode(p) **TODO: Clarify how is this ordered.**
- ORDER BY totalAmount(p) or ORDER BY endTime(p)
- user-defined path-global functions.

TODO: How are nodes IDs ordered? Do we just assume identifier ordering?

Expressions over internal path elements are not directly available unless the path has first been decomposed.

2.4 Framing

Frame over scalar, node, or edge bindings. This computes a rolling sum over matched transfer edges, grouped by currency.

```

MATCH (a:Account)-[t:TRANSFER]->(b:Account)
RETURN a.name AS source, b.name AS target, t.time AS time,
       t.amount AS amount, t.currency AS currency,
       sum(t.amount) OVER (
         PARTITION BY t.currency ORDER BY t.time
         RANGE BETWEEN INTERVAL '30' DAY PRECEDING AND CURRENT ROW
       ) AS rolling_30d_outflow

```

Listing 20: Rolling transfer amount over ordinary binding rows.

Frame over path bindings. In this example, the input relation contains one binding per matched path. The window partitions this relation of path bindings by the endpoint pair (a, b) .

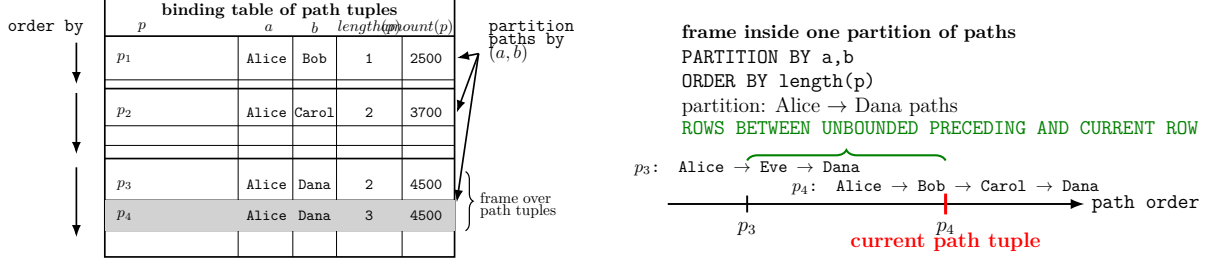


Figure 3: **Left:** The frame ranges over path tuples in the same partition. It does not range over the internal nodes or relationships of a path. Internal path iteration requires a separate **OVER PATH** semantics. **Right:** The binding table contains path tuples. The window partitions the relation of paths, for example by the endpoint pair (a, b) . The partition is not the path object itself; the path object is the tuple being partitioned.

source	target	time	currency	amount	rolling_30d_outflow
Alice	Bob	2026-01-05 09:15:00	EUR	2500	2500
Bob	Carol	2026-01-07 14:30:00	EUR	1200	3700
Carol	Dana	2026-01-09 11:45:00	EUR	800	4500
Alice	Eve	2026-01-06 16:20:00	USD	3000	3000
Eve	Dana	2026-01-10 10:05:00	USD	1500	4500

Table 7: Result of the rolling transfer query in Listing 20.

The frame ranges over path bindings in the same partition. It does not range over the internal nodes or relationships of any individual path.

```

MATCH p = (a:Account)-[:TRANSFER*]->(b:Account)
RETURN a.name AS source, b.name AS target,
       rank() OVER (PARTITION BY a.name, b.name
                    ORDER BY length(p) ASC)
AS path_length_rank

```

Listing 21: Ranking path bindings by a path-global expression.

source	target	path_length_rank
Alice	Bob	1
Bob	Carol	1
Carol	Dana	1
Alice	Eve	1
Eve	Dana	1
Alice	Carol	1
Bob	Dana	1
Alice	Dana	1
Alice	Dana	2

Table 8: Result of the path-binding ranking query in Listing 21.

For the pair (Alice,Dana), there are two paths. The shorter path, Alice → Eve → Dana, receives rank 1; the longer path, Alice → Bob → Carol → Dana, receives rank 2. The window does not see the internal nodes or relationships of the path. It can rank path bindings by a path-global expression, but it cannot express predicates over internal path fragments such as consecutive triples, edge amounts, or local temporal constraints. For instance, `length(p)` treats the path as an atomic value with a global property.

Limitation. Alternative 1 is row-based even when the rows contain paths. It can rank, aggregate, or compare path bindings using path-global expressions, but it cannot express conditions over consecutive edges, internal nodes, or local path fragments. Such queries require `OVER PATH` or a path-scoped window match.

2.5 Path-element framing

Alternative 1 treats a bound path as an atomic value inside a binding tuple. This is sufficient for window functions over path-level properties, such as `length(p)`, `startNode(p)`, `endNode(p)`, or `totalCost(p)`. However, it does not allow the frame to range over the internal nodes or edges of the path.

We therefore consider a path-element framing extension. The extension first unfolds a bound path into a derived relation of path-element occurrences. The window function is then evaluated over this derived relation using ordinary row-based window semantics. Thus, the extension remains compatible with Alternative 1, but changes the row domain from path bindings to path-element bindings.

draft Idea is unfolding the path and allowing the window to frame within its content, edges or nodes.

```
MATCH p = (s:Station)-[:LINK*]->(t:Station)
RETURN
  s,
  t
  p,
  aggregation(e.distance) OVER EDGES OF PATH(p) (
    PARTITION BY s,t
    ORDER BY POSITION(e)
    EDGES BETWEEN UNBOUNDED PRECEDING AND CURRENT EDGE
  ) AS cumulative_distance
```

```
MATCH p = (s:Station)-[:LINK*]->(t:Station)
RETURN
  s,
  t
  p,
  aggregation(e.distance) OVER NODES OF PATH(p) (
    PARTITION BY s,t
    ORDER BY POSITION(e)
    NODES BETWEEN UNBOUNDED PRECEDING AND CURRENT NODEs
  ) AS cumulative_distance
```

This creates a new binding table for each elements (edges or nodes). I think this is closely related to range and groups see below

```
MATCH p = (s:Station)-[:LINK*]->(t:Station)
RETURN
  s,
  t
  p,
  aggregation(e.distance) OVER EDGES OF PATH (
    PARTITION BY s,t
    ORDER BY POSITION(e)
    RANGE PATH BETWEEN UNBOUNDED PRECEDING AND CURRENT EDGE
  ) AS cumulative_distance
```

```

MATCH p = (s:Station)-[:LINK*]->(t:Station)
RETURN
  s,
  t
  p,
  aggregation(e.distance) OVER NODE OF PATH(p) (
    PARTITION BY s,t
    ORDER BY POSITION(e)
    GROUP PATH BETWEEN UNBOUNDED PRECEDING AND CURRENT NODE
  ) AS cumulative_distance

```

Alternatively, the ROW based equivalent in the graph alternative 1 is using a MATCH clause where CURRENT is either a node or edge based on what are we iterating one.

```

MATCH p = (s:Station)-[:LINK*]->(t:Station)
RETURN
  s,
  t
  p,
  aggregation(e.distance) OVER EDGES OF PATH(p) (
    PARTITION BY s,t
    ORDER BY POSITION(e)
    BETWEEN MATCH (a) -[CURRENT EDGE] -> (b)
  ) AS cumulative_distance

```

```

MATCH p = (s:Station)-[:LINK*]->(t:Station)
RETURN
  s,
  t
  p,
  aggregation(e.distance) OVER NODEs OF PATH(p) (
    PARTITION BY s,t
    ORDER BY POSITION(e)
    BIND AS (a) -[x]->(CURRENT NODE) -> (b)
  ) AS cumulative_distance

```

Questions?

startig from WALK semantics, can we reduce it to TRIAL or SIMPLE or SHORTEST using window function in altenative one? this would give the user less burden

Path-element framing.

add hat

Listing 22: Path-element framing over edge occurrences.

```

MATCH p = (s:Station)-[:LINK*]->(t:Station)
RETURN
  s,
  t,
  p,
  sum(e.distance) OVER PATH p EDGES AS e (
    PARTITION BY p
    ORDER BY position(e)
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
  ) AS cumulative_distance

```

Listing 22 unfolds each path p into one row per edge occurrence. The partition key is p, so the cumulative distance is computed independently inside each path. The ordering expression

path	position	edge	distance
p_1	1	$A \rightarrow B$	5
p_1	2	$B \rightarrow C$	7
p_1	3	$C \rightarrow D$	2

Table 9: Edge-occurrence relation induced by Listing 22.

path	position	edge	distance	cumulative_distance
p_1	1	$A \rightarrow B$	5	5
p_1	2	$B \rightarrow C$	7	12
p_1	3	$C \rightarrow D$	2	14

Table 10: Prefix-distance result for the edge frame in Listing 22.

`position(e)` orders edge occurrences according to their position in the path. The frame is a prefix frame over edge occurrences.

For a path

$$p_1 = A \xrightarrow{5} B \xrightarrow{7} C \xrightarrow{2} D,$$

the edge-unfolded relation is:

The corresponding cumulative result is:

The same idea applies to node occurrences.

Listing 23: Path-element framing over node occurrences.

```
MATCH p = (s:Station)-[:LINK*]->(t:Station)
RETURN
  s,
  t,
  p,
  sum(n.load) OVER PATH p NODES AS n (
    PARTITION BY p
    ORDER BY position(n)
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
  ) AS cumulative_load
```

Listing 23 unfolds each path into one row per node occurrence. The window frame ranges over node occurrences of the same path, ordered by their path position.

Relation to ROWS, RANGE, and GROUPS. Path-element framing should reuse the standard SQL frame modes where possible. The novelty is not the frame mode itself, but the domain over which the frame is evaluated.

For example, a ROWS-style path frame over edges is:

Listing 24: Offset-based frame over edge occurrences.

```
MATCH p = (s:Station)-[:LINK*]->(t:Station)
RETURN
  s,
  t,
  p,
  sum(e.distance) OVER PATH p EDGES AS e (
    PARTITION BY p
    ORDER BY position(e)
    ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
  ) AS local_distance
```

SQL frame mode	Path-element analogue	Meaning
ROWS	element offsets	previous or following node/edge occurrences
RANGE	value offsets	elements within an ordered value interval
GROUPS	peer groups	groups of elements with equal order values

Table 11: Relationship between classical frame modes and path-element framing.

A RANGE-style path frame is value-based. For instance, it may use the cumulative distance of the current edge as the ordering value:

Listing 25: Range-based frame over edge occurrences.

```

MATCH p = (s:Station)-[:LINK*]->(t:Station)
RETURN
  s,
  t,
  p,
  sum(e.distance) OVER PATH p EDGES AS e (
    PARTITION BY p
    ORDER BY cumulativeDistance(e)
    RANGE BETWEEN 10 PRECEDING AND CURRENT ROW
  ) AS distance_last_10_units

```

Listing 25 includes, for each current edge occurrence, the edge occurrences of the same path whose cumulative path-distance values lie within ten units before the current edge.

A GROUPS-style path frame is peer-group based:

Listing 26: Group-based frame over node occurrences.

```

MATCH p = (s:Station)-[:LINK*]->(t:Station)
RETURN
  s,
  t,
  p,
  count(n) OVER PATH p NODES AS n (
    PARTITION BY p
    ORDER BY n.zone
    GROUPS BETWEEN 1 PRECEDING AND CURRENT ROW
  ) AS count_current_and_previous_zone

```

Here, node occurrences with the same `zone` value are peers. The frame contains the current peer group and the immediately preceding peer group.

Path-scoped pattern frames. A separate extension is needed when the frame is defined by a local pattern around the current path element. This should not be encoded inside `BETWEEN`. Instead, the pattern should be introduced as an explicit path-scoped frame condition.

Listing 27: Path-scoped pattern frame around the current edge.

```

MATCH p = (s:Station)-[:LINK*]->(t:Station)
RETURN
  s,
  t,
  p,
  sum(e.distance) OVER PATH p EDGES AS e (
    PARTITION BY p
    ORDER BY position(e)
    FRAME MATCH (a)-[e]->(b) WITHIN p
  )

```

```
) AS local_edge_value
```

For node-centered frames:

Listing 28: Path-scoped pattern frame around the current node.

```
MATCH p = (s:Station)-[:LINK*]->(t:Station)
RETURN
  s,
  t,
  p,
  sum(x.distance) OVER PATH p NODES AS n (
    PARTITION BY p
    ORDER BY position(n)
    FRAME MATCH (a)-[x]->(n)-[y]->(b) WITHIN p
  ) AS local_node_context
```

The distinction is important. The construct `OVER PATH p EDGES AS e` defines the iteration domain. The construct `FRAME MATCH ... WITHIN p` defines a local path-scoped pattern around the current element.

Cardinality. Path unfolding changes the result cardinality. Since each path produces one row per edge or node occurrence, the query must specify how the result is returned. There are three natural modes.

1. **Row-emitting mode.** The query returns one row per path element.

```
RETURN s, t, p, e, cumulative_distance
```

2. **Vector mode.** The query returns one row per path and collects the values in path order.

```
RETURN s, t, p,
       collect(cumulative_distance ORDER BY position(e)) AS distances
```

3. **Annotated-path mode.** The query returns one path value enriched with computed element values.

```
RETURN s, t, annotate(p, e, cumulative_distance) AS annotated_path
```

Without this distinction, the semantics is ambiguous.

Path modes as reductions over unfolded paths. Path-element framing also provides a denotational way to describe common path modes. Starting from a finite relation of walk bindings, trail, simple, and shortest-path semantics can be expressed as windowed predicates over unfolded path elements.

A trail is a walk with no repeated edge:

Listing 29: Trail semantics using two window phases and `QUALIFY`.

```
MATCH p = WALK (s)-[:LINK*]->(t)
WITH
  p,
  count(e) OVER PATH p EDGES AS e (
    PARTITION BY p, e
  ) AS edge_occurrences
RETURN p
QUALIFY
  max(edge_occurrences) OVER (
    PARTITION BY p
  ) = 1
```

A simple path is a walk with no repeated node:

Listing 30: Simple-path semantics using two window phases and QUALIFY.

```
MATCH p = WALK (s)-[:LINK*]->(t)
WITH
  p,
  count(n) OVER PATH p NODES AS n (
    PARTITION BY p, n
  ) AS node_occurrences
RETURN p
QUALIFY
  max(node_occurrences) OVER (
    PARTITION BY p
  ) = 1
```

Shortest-path semantics can be described as a ranking over a finite path relation:

Listing 31: Shortest-path semantics using ranking and QUALIFY.

```
MATCH p = WALK (s)-[:LINK*]->(t)
RETURN p
QUALIFY
  rank() OVER (
    PARTITION BY s, t
    ORDER BY length(p)
  ) = 1
```

For weighted shortest paths, `length(p)` can be replaced by a path-global cost expression such as `totalCost(p)`.

These reductions are useful as denotational semantics, but they are not sufficient as execution strategies. In cyclic graphs, the set of walks can be infinite. Therefore, the reductions are sound only when the candidate walk relation is finite, for example because of a hop bound, an acyclicity condition, or a built-in finite path mode.

Path modes such as `TRAIL`, `SIMPLE`, and `SHORTEST` should remain user-facing primitives. They can be explained denotationally as windowed predicates over unfolded path-element relations, but they should not be implemented by enumerating all walks and filtering afterwards.

2.6 CTEs and graph bindings windows.

In the setting described above, a CTE can be best understood as a named binding relation. A graph pattern first produces rows of bindings. These rows may contain nodes, edges, paths, or scalar values. A CTE can name this relation before a later windowing or filtering phase.

Listing 32: Filtering a window result using Cypher-style query boundaries.

```
MATCH p = (a:Account)-[:TRANSFER*]->(b:Account)
WITH
  p,
  a,
  b,
  RANK() OVER (
    PARTITION BY a.name, b.name
    ORDER BY path_length
  ) AS path_rank
WITH
  p,
```

```

a,
b,
length(p) AS path_length
WHERE path_rank = 1
RETURN
a.name AS source,
b.name AS target,
p,
path_rank

```

Listing 32 illustrates the relation between CTEs and the row-based graph-window alternative. The CTE does not define a new graph frame. It names the binding table over which a row-based window is evaluated.

2.7 Strengths

- Familiar to SQL users.
- Clear cardinality: rows are preserved.
- Natural for top- k neighbors, rankings, temporal graph events, deduplication, and rolling statistics.
- Optimizers can reuse relational window-operator machinery after pattern matching.

2.8 Invariants

This alternative satisfies the classical invariants of row-based window functions.

1. **binding preservation.** The window operator appends computed values to the binding bindings; it does not filter, duplicate, or collapse them.
2. **Partition coverage.** Every binding belongs to exactly one partition for a given window specification. If no `PARTITION BY` clause is given, all bindings belong to one global partition.
3. **Partition disjointness.** Partitions do not overlap. A binding cannot belong to two different partitions under the same window specification.
4. **Partition-local access.** A binding can access only bindings in its own partition.
5. **Frame containment.** For every binding r , $\text{frame}(r) \subseteq \text{partition}(r)$.
6. **Ordering locality.** The `ORDER BY` clause defines the logical order inside each partition, not necessarily the final output order.
7. **Current-binding relativity.** Frames are computed relative to the current binding. Different bindings in the same partition may have different frames.
8. **Peer consistency.** bindings with the same `ORDER BY` value inside a partition are peers. Peer-sensitive functions such as `rank()` treat peers uniformly.
9. **Function-dependent framing.** Aggregate window functions such as `sum()` and `avg()` are frame-sensitive, while ranking functions such as `rank()` and `row_number()` are evaluated over the partition order and ignore the frame.
10. **No implicit structural descent.** If a binding contains a graph object such as a node, edge, or path, the window sees that object as a binding attribute. It does not automatically descend into the object's internal structure.

2.9 Limitations

- It does not directly express movement along a path.
- It cannot naturally express a moving graph neighborhood.
- Path multiplicity can affect counts unless explicitly deduplicated.

- It exposes only path-global expressions unless an explicit path-decomposition operator is introduced.